# Atelier R pour projets d’etude et de recherche

Repères pour utilisateurs de Stata

J. Montornes

08/05/2026

# Objectifs de l'atelier

- Comprendre les differences importantes entre R et Stata.
- Ecrire un script reproductible plutot qu'une suite de commandes isolees.
- Manipuler des donnees avec `dplyr`.
- Produire graphiques, regressions et sorties simples.
- Savoir ou chercher de l'aide et comment structurer un projet.

# Deroule

1. Interface, aide et organisation du projet.
2. Packages, objets et syntaxe.
3. Importation et manipulation des donnees.
4. Graphiques et tableaux.
5. Estimation econometrique.
6. Dates, chaines de caracteres et exercice recapitulatif.

# Interface RStudio

- **Script** : code source conserve, versionnable et relancable.
- **Console** : execution interactive et messages d'erreur.
- **Environment** : objets crees en memoire.
- **Files / Plots / Packages / Help** : sorties, fichiers, documentation.

Reflexe a prendre : tout ce qui doit etre garde va dans un script, pas seulement dans la console.

# Aide et documentation

```
?lm
help("merge")
example(mean)
```

Ressources utiles :

- UtilitR
- Stata2R
- R for Data Science

# Packages

Un package ajoute des fonctions, des jeux de donnees ou des methodes.

```
install.packages("dplyr")  # une fois par machine
library(dplyr)             # a chaque nouvelle session
```

Pour une formation, indiquer les packages requis dans le README ou dans un script d'installation evite les surprises le jour J.

# Objets et assignation

```
prix_baguette <- 0.95
quantites <- c(1, 2, 3)
depense <- prix_baguette * quantites
depense
```

```
## [1] 0.95 1.90 2.85
```

En R, on assigne presque toujours un resultat a un objet avec `<-`. Le signe `=` fonctionne souvent, mais `<-` reste la convention la plus lisible.

# Differences utiles avec Stata

| Stata | R |
|---|---|
| Une base active en memoire | Plusieurs objets simultanement |
| **use**, **save** | `read.csv()`, `write.csv()`, `readxl::read_xlsx()` |
| **gen**, **replace** | `mutate()` |
| **keep if** | `filter()` |
| **collapse** | `group_by()` puis `summarise()` |
| **merge** | `left_join()`, `inner_join()`, `full_join()` |

# Importer un fichier

```r
economy <- read.csv("data/sample_economy.csv")
head(economy)
```

```
##   country year      sector employment wage price_index consumption
## 1  France 2020    Industry       3100 37.2       101.4       220.5
## 2  France 2020    Services       8200 34.8       101.4       516.2
## 3  France 2020 Agriculture        640 25.1       101.4        41.0
## 4  France 2021    Industry       3180 38.1       103.1       229.8
## 5  France 2021    Services       8460 35.7       103.1       538.4
## 6  France 2021 Agriculture        630 25.8       103.1        42.3
```

Pour Excel :

```r
readxl::read_xlsx("data/mon_fichier.xlsx", sheet = "Feuil1")
```

# Selectionner et filtrer

## Stata

```
keep if country == "France" & year >= 2021
keep country year sector wage
```

## R

```
economy |>
  filter(country == "France", year >= 2021) |>
  select(country, year, sector, wage)
```

```
##   country year      sector wage
## 1  France 2021    Industry 38.1
## 2  France 2021    Services 35.7
## 3  France 2021 Agriculture 25.8
## 4  France 2022    Industry 40.0
## 5  France 2022    Services 37.6
## 6  France 2022 Agriculture 27.2
```

# Creer une variable

## Stata

```
gen real_wage = wage / price_index * 100
```

## R

```
economy <- economy |>
  mutate(real_wage = wage / price_index * 100)

economy |>
  select(country, year, sector, real_wage) |>
  head()
```

```
##   country year      sector real_wage
## 1  France 2020    Industry  36.68639
## 2  France 2020    Services  34.31953
## 3  France 2020 Agriculture  24.75345
## 4  France 2021    Industry  36.95441
## 5  France 2021    Services  34.62658
## 6  France 2021 Agriculture  25.02425
```

# Agreger

## Stata

```
collapse (mean) mean_wage = wage, by(country year)
```

## R

```
country_year <- economy |>
  group_by(country, year) |>
  summarise(mean_wage = mean(wage), .groups = "drop")

country_year
```

```
## # A tibble: 9 × 3
##   country  year mean_wage
##   <chr>   <int>     <dbl>
## 1 France   2020      32.4
## 2 France   2021      33.2
## 3 France   2022      34.9
## 4 Germany  2020      33.4
## 5 Germany  2021      34.1
## 6 Germany  2022      36.0
## 7 Italy    2020      27.8
## 8 Italy    2021      28.5
## 9 Italy    2022      30.0
```

# Fusionner deux tables

```r
population <- data.frame(
  country = c("France", "Germany", "Italy"),
  population_m = c(67.8, 83.2, 59.0)
)

country_year |>
  left_join(population, by = "country") |>
  head()
```

```
## # A tibble: 6 × 4
##   country  year mean_wage population_m
##   <chr>   <int>     <dbl>        <dbl>
## 1 France   2020      32.4         67.8
## 2 France   2021      33.2         67.8
## 3 France   2022      34.9         67.8
## 4 Germany  2020      33.4         83.2
## 5 Germany  2021      34.1         83.2
## 6 Germany  2022      36.0         83.2
```

Retenir le sens de la jointure : `left_join(x, y)` garde toutes les lignes de `x`.

# data.table en une phrase

`data.table` est tres efficace sur de gros volumes. Sa syntaxe est compacte :

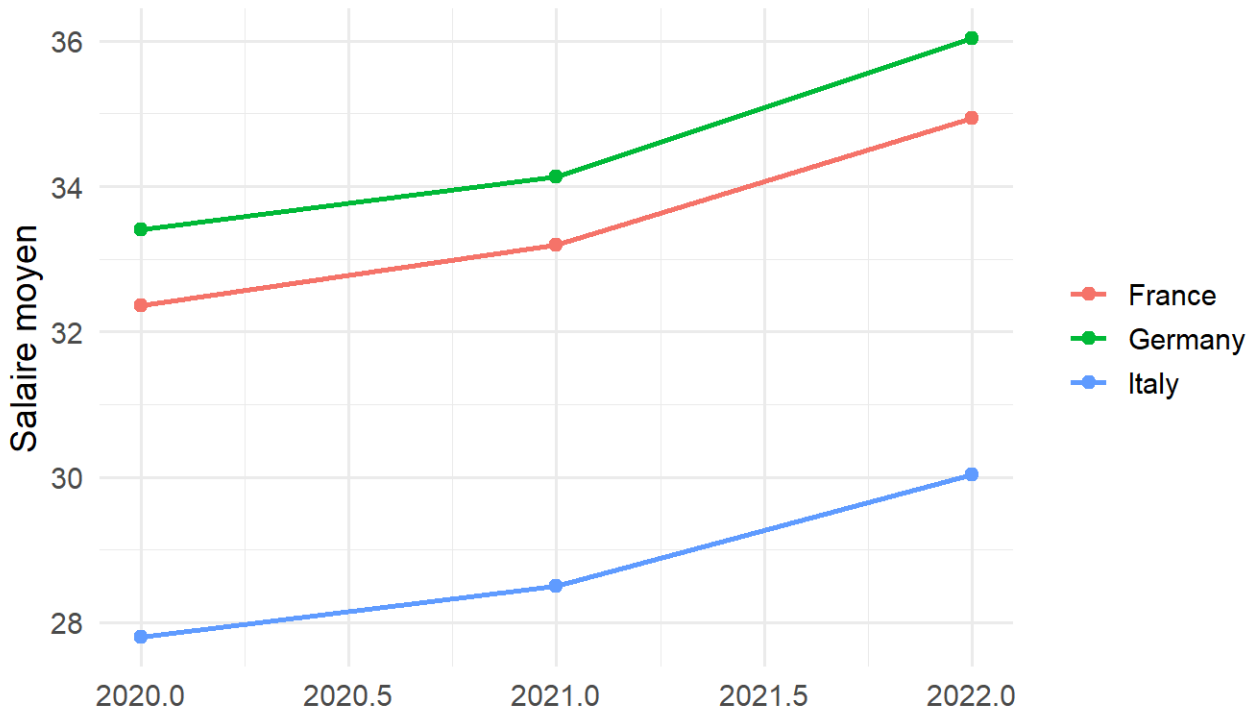
```
library(data.table)
dt <- as.data.table(economy)
dt[country == "France", .(mean_wage = mean(wage)), by = year]
```

```
##     year mean_wage
##    <int>     <num>
## 1:  2020  32.36667
## 2:  2021  33.20000
## 3:  2022  34.93333
```

Pour debuter, `dplyr` est souvent plus lisible ; pour des bases massives, `data.table` peut devenir preferable.

# Graphique simple

```
ggplot(country_year, aes(x = year, y = mean_wage, color = country)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  labs(x = NULL, y = "Salaire moyen", color = NULL)
```



# Principes ggplot2

- `ggplot(data, aes(...))` declare les donnees et les variables.
- `geom_*()` choisit la representation : points, lignes, barres.
- `labs()`, `scale_*()` et `theme_*()` ajustent la presentation.
- Chaque couche s'ajoute avec `+`.

```
ggplot(data, aes(x = x, y = y)) +
  geom_point() +
  labs(x = "X", y = "Y")
```

# Regression lineaire

## Stata

```
reg real_wage employment i.sector
```

## R base

```
model_lm <- lm(real_wage ~ employment + factor(sector), data = economy)
summary(model_lm)
```

```
##
## Call:
## lm(formula = real_wage ~ employment + factor(sector), data = economy)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -3.8674 -1.6908  0.4735  1.4333  2.7922
##
## Coefficients:
##                          Estimate Std. Error t value Pr(>|t|)
## (Intercept)            2.324e+01  7.366e-01  31.553  < 2e-16 ***
## employment             7.674e-04  2.274e-04   3.375  0.00261 **
## factor(sector)Industry 8.481e+00  1.430e+00   5.932 4.78e-06 ***
## factor(sector)Services 2.559e+00  2.291e+00   1.117  0.27562
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.164 on 23 degrees of freedom
## Multiple R-squared:  0.8762, Adjusted R-squared:   0.86
## F-statistic: 54.25 on 3 and 23 DF,  p-value: 1.372e-10
```

# Effets fixes avec fixest

`fixest` est proche de l'esprit de `reghdfe` : rapide, concis, pratique pour les effets fixes et les erreurs standards groupees.

```
model_fe <- feols(
  real_wage ~ employment | country + year,
  data = economy,
  vcov = ~ country
)
etable(model_fe)
```

```
##                           model_fe
## Dependent Var.:          real_wage
##
## employment      0.0010** (4.85e-5)
## Fixed-Effects:  ------------------
## country                        Yes
## year                           Yes
## _______________ __________________
## S.E.: Clustered        by: country
## Observations                    27
## R2                         0.61545
## Within R2                  0.54274
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

# Sortir les resultats proprement

```
broom::tidy(model_lm)
```

```
## # A tibble: 4 × 5
##   term                    estimate std.error statistic  p.value
##   <chr>                      <dbl>     <dbl>     <dbl>    <dbl>
## 1 (Intercept)            23.2       0.737        31.6  1.94e-20
## 2 employment              0.000767  0.000227      3.37 2.61e- 3
## 3 factor(sector)Industry  8.48      1.43          5.93 4.78e- 6
## 4 factor(sector)Services  2.56      2.29          1.12 2.76e- 1
```

Pour un rapport :

- garder les objets de modele (`model_lm`, `model_fe`) ;
- produire les tableaux dans le script ;
- eviter les copier-coller manuels depuis la console.

# Dates avec lubridate

```
jour <- dmy("11 fevrier 2023", locale = Sys.getlocale("LC_TIME"))
jour
```

```
## [1] NA
```

```
year(jour)
```

```
## [1] NA
```

```
month(jour)
```

```
## [1] NA
```

`ymd()`, `dmy()` et `mdy()` clarifient l'ordre jour/mois/annee.

# Chaines de caracteres avec stringr

```r
texte <- "Hello world"
str_replace(texte, "world", "universe")
```

```
## [1] "Hello universe"
```

```r
str_sub(texte, 1, 5)
```

```
## [1] "Hello"
```

```r
str_detect(texte, "Hello")
```

```
## [1] TRUE
```

Equivalent mental :

- Stata `subinstr()` -> R `str_replace()` ou `str_replace_all()`.
- Stata `substr()` -> R `str_sub()`.
- Stata `regexm()` -> R `str_detect()`.

# Appels web et bases externes

Pour DB.NOMICS ou d'autres API, presenter le code sans l'executer dans les slides rend le support robuste.

```
library(rdbnomics)
df <- rdbnomics::rdb(ids = "INSEE/CNT-2014-CSI/...")
```

En formation, prevoir un fichier local de secours si le reseau, le proxy ou l'API ne repond pas.

# Mini-projet recommande

1. Importer `data/sample_economy.csv`.
2. Creer un salaire reel.
3. Calculer un salaire moyen par pays et annee.
4. Tracer l'evolution temporelle.
5. Estimer un modele simple avec effets fixes.
6. Sauvegarder le graphique et la table de resultats.

# Bonnes pratiques de projet

```
AtelierR/
  data/
  exercises/
  solutions/
  atelierR.Rmd
  atelierR_en.Rmd
  README.md
```

- Chemins relatifs depuis la racine du projet.
- Donnees d'exemple legeres dans `data/`.
- Code de correction executable dans `solutions/`.
- README avec objectifs, pre-requis et commandes de rendu.

# Conclusion

R n’est pas seulement une calculatrice statistique : c’est un environnement de projet.

Le gain principal par rapport a un usage interactif est la reproductibilite : demain, quelqu’un doit pouvoir relancer le script et obtenir les memes sorties.